# StudyBuddy

**AI Study Coach Agent**

An AI-agent web application built with Node.js, Express and the Google Gemini API

**Live application**

https://studybuddy-XXXX.vercel.app

**Source code repository**

https://github.com/manganjokean/studybuddy


**Author:** FULL NAME
**Student ID:** STUDENT ID

**Course:** Web Client-Side Programming
**Instructor:** INSTRUCTOR NAME
**Date:** October 29, 2026

**Abstract**

StudyBuddy is a publicly deployed web application that helps students turn their own lecture notes into interactive study material. The project implements an *AI agent* rather than a plain chatbot: a Node.js/Express back end exposes a set of tools to the Google Gemini model, and an agentic loop lets the model decide which tool to call, execute it on the server, feed the result back, and only then produce a final answer for the user. The client side is a chat interface written in vanilla HTML, CSS and JavaScript: user and agent turns appear as bubbles, the agent's tool calls are animated as a visible plan, and the quiz, flashcards or explanation built by a tool are rendered inline in the same conversation. Quiz results are persisted in the browser with `localStorage`. The application is deployed on Vercel and the Gemini API key is kept entirely outside the repository through environment variables, a strict `.gitignore` and an automated pre-push secret scanner. This report describes the requirements, architecture, agent design, implementation, deployment, security measures, testing, and lessons learned during the project.

# Contents

1 Introduction 2

2 Requirements 2

3 Technology Stack 3

4 System Architecture 3

5 AI Agent Design 4

6 Client-Side Implementation 5

7 Deployment 6

8 Security: API Key Handling 7

9 Testing 7

10 Challenges and Lessons Learned 7

11 Conclusion 8

# 1 Introduction

Students routinely revise from passive material: re-reading slides, highlighting text and re-reading again. Research on learning consistently favours *active recall* — retrieving information from memory — but producing practice questions from one's own notes is tedious.

StudyBuddy addresses this with a small, focused web application. A student pastes their notes (or types a topic), and an AI agent decides how best to help: it can generate a quiz, build a flashcard deck, or explain a concept at an appropriate level. The distinctive feature of the project is that the model does not simply emit text: it *calls tools* that the server executes, and the structured output of those tools drives the interactive front end.

### Project objectives

- Build a working web application whose front end demonstrates client-side skills (DOM manipulation, `fetch`, event handling, state, local persistence, responsive CSS).
- Implement an AI agent with a real tool-calling loop, not a single chat completion.
- Deploy the application to a public host so it can be opened by anyone.
- Keep the API key out of source control and out of the browser.
- Produce this report, linked to the live application and the source repository.
- Present and demonstrate the application in class.

# 2 Requirements

Table 1: Functional and non-functional requirements.

| ID | Requirement | Status |
|---|---|---|
| FR-1 | Accept study material or a free-text topic from the user | Implemented |
| FR-2 | Agent selects and executes at least two distinct tools | Implemented (3 tools) |
| FR-3 | Display the agent's plan / tool-call trace to the user | Implemented |
| FR-4 | Interactive quiz with per-question feedback and a final score | Implemented |
| FR-5 | Flashcard deck with flip interaction | Implemented |
| FR-6 | Structured concept explanation | Implemented |
| FR-7 | Persist quiz history locally in the browser | Implemented (`localStorage`) |
| FR-8 | Graceful error handling for invalid input, rate limits and API failures | Implemented |
| NFR-1 | Deployed to a public URL | Vercel |
| NFR-2 | No API key in the repository or in client-side code | Enforced by tooling |
| NFR-3 | Responsive layout, no front-end framework | Vanilla HTML/CSS/JS |

## 3 Technology Stack

Table 2: Technologies used.

| Layer | Technology | Reason |
|---|---|---|
| Front end | HTML5, CSS3, vanilla ES6 JS | Course focus: client-side programming |
| Back end | Node.js 22+ / Express 5 | Simple static + JSON API server |
| AI model | Google Gemini (`gemini-3.8-flash`) | Native function calling, free tier |
| API style | REST (`generateContent`) | No SDK dependency, transparent requests |
| Hosting | Vercel | Static files + Node serverless functions, free tier |
| Versioning | Git / GitHub | Required repository link |
| Report | LaTeX on Overleaf | PDF output (required deliverable) |

## 4 System Architecture

The application is a classic two-tier structure. The browser never talks to the AI provider: all model calls go through the back end – an Express server during local development, a Vercel serverless function in production – which is the only component that holds the API key.

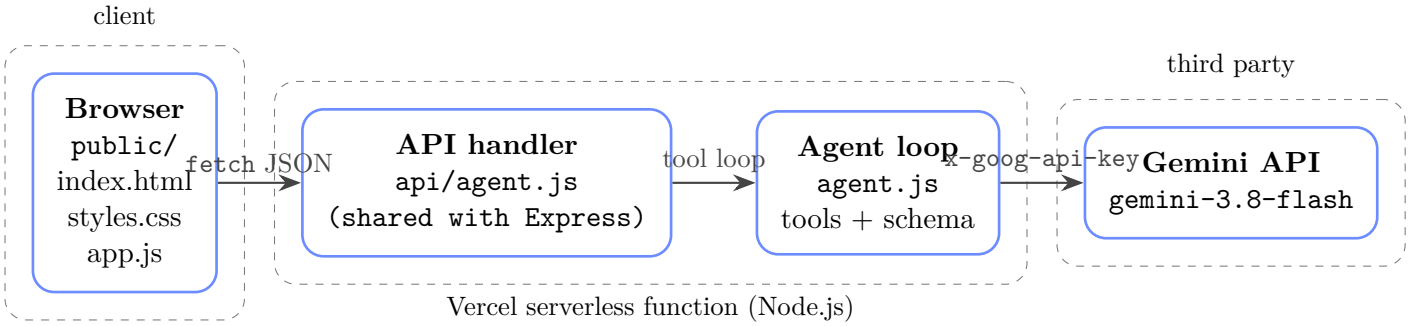



Figure 1: Request flow. The API key exists only inside the Vercel environment (or the developer's local `.env`).

### Project structure

```
studybuddy/
|-- server.js            # Express server for local dev, mounts ./api
|-- agent.js             # agent loop, tool schemas, tool implementations
|-- api/
|   |-- agent.js         # POST /api/agent   (Vercel function + Express route)
|   `-- health.js        # GET  /api/health  (Vercel function + Express route)
|-- vercel.json          # Vercel config: static output public/, 30s limit
|-- scripts/keycheck.js  # pre-push secret scanner
|-- scripts/shoot.mjs    # headless UI test + report screenshots
|-- public/
|   |-- index.html       # markup: chat thread, composer, artefact cards
|   |-- styles.css       # theming, quiz UI, 3D flip cards, responsive
|   `-- app.js           # fetch, trace animation, quiz logic, storage
```

```
|-- report/              # this LaTeX report + figures
|-- .env                 # API key (gitignored, never pushed)
`-- .env.example         # placeholder committed on purpose
```

# 5 AI Agent Design

## What makes it an agent

A chatbot performs a single request/response and returns prose. StudyBuddy's back end instead runs a *loop*:

1. The user goal, conversation history and a system prompt are sent to Gemini together with a list of **function declarations** (the tool schemas).
2. If the model answers with a `functionCall`, the server executes that tool locally, appends a `functionResponse` containing the real result, and calls the model again.
3. Steps repeat until the model replies with plain text, or a safety limit of four rounds is reached.
4. The response returned to the browser contains three parts: the agent's natural-language reply, the **trace** of tools invoked, and the raw structured **results** used to render the interactive components.

Listing 1: Simplified agent loop (agent.js).

```
for (let step = 0; step < MAX_STEPS; step++) {
  const parts = await callGemini({ contents, tools: TOOLS,
                                   systemInstruction: SYSTEM_PROMPT });
  const calls = parts.filter(p => p.functionCall);
  if (!calls.length) { finalText = joinText(parts); break; }  // done

  contents.push({ role: 'model', parts });
  for (const c of calls) {
    const output = await TOOL_IMPLS[c.functionCall.name](c.functionCall.args);
    results[c.functionCall.name] = output;                    // structured result
    contents.push({ role: 'user', parts: [{ functionResponse: ... }] });
  }
}
```

## Available tools

## Robustness measures

- **Structured output**: quizzes and flashcards are requested as JSON with an explicit schema, and every returned field is validated before it reaches the browser, instead of trusting the model to format things correctly.
- **Input clamping**: question counts, string lengths and history size are bounded before they reach the API.
- **Graceful degradation**: if the final summarising call fails (for example due to a rate limit), the already-produced structured results are still returned with a fallback message, so the user is not left with an error screen.

Table 3: Tools exposed to the model.

| Tool | Arguments | Behaviour |
|---|---|---|
| `generateQuiz` | `source, topic, count, difficulty` | Calls Gemini with a JSON response schema; validates each question and clamps `count` to 3–10 |
| `generateFlashcards` | `source, topic, count` | Produces front/back card pairs, filtered for well-formed entries |
| `explainConcept` | `concept, level` | Free-text answer forced into `##` sections so the client can render it reliably |

- **Friendly errors**: HTTP 429, 400/403 and network failures are mapped to plain-language messages; technical details are written to the server log only.

## 6 Client-Side Implementation

The graded focus of the course is client-side development, so the front end is written without any framework.

- **Conversation thread**: user and agent turns are rendered as chat bubbles with a typing indicator while the agent works, and the thread auto-scrolls to the newest message.
- **Agent trace animation**: each tool call returned by the API is appended to an "Agent plan" block one step at a time, so the audience can watch the agent decide and act before the answer arrives.
- **Inline artefact cards**: the quiz, flashcard deck or explanation produced by a tool is injected directly into the same conversation turn – no page change, no separate results screen.
- **Quiz engine**: one question at a time, immediate green/red feedback together with the explanation, a progress pill, and a conic-gradient score ring at the end.
- **Flashcards**: a 3D `rotateY` flip implemented with `transform-style: preserve-3d` and `backface-visibility`, plus a Fisher–Yates shuffle.
- **State & persistence**: scores are written to `localStorage` under `studybuddy.scores`; the last eight conversation turns are sent with every request so the agent can answer follow-ups in context.
- **Composer**: *Enter* sends, *Shift+Enter* inserts a newline, the textarea grows with its content, and mode/level/item selectors steer the agent's tool choice.
- **Resilience**: spinner on the send button, toast notifications for errors, and all dynamic text inserted through an HTML-escaping helper to avoid injection from model output.
- **Responsive layout**: a single-column chat shell that adapts below 780 px, with touch-friendly card grids.

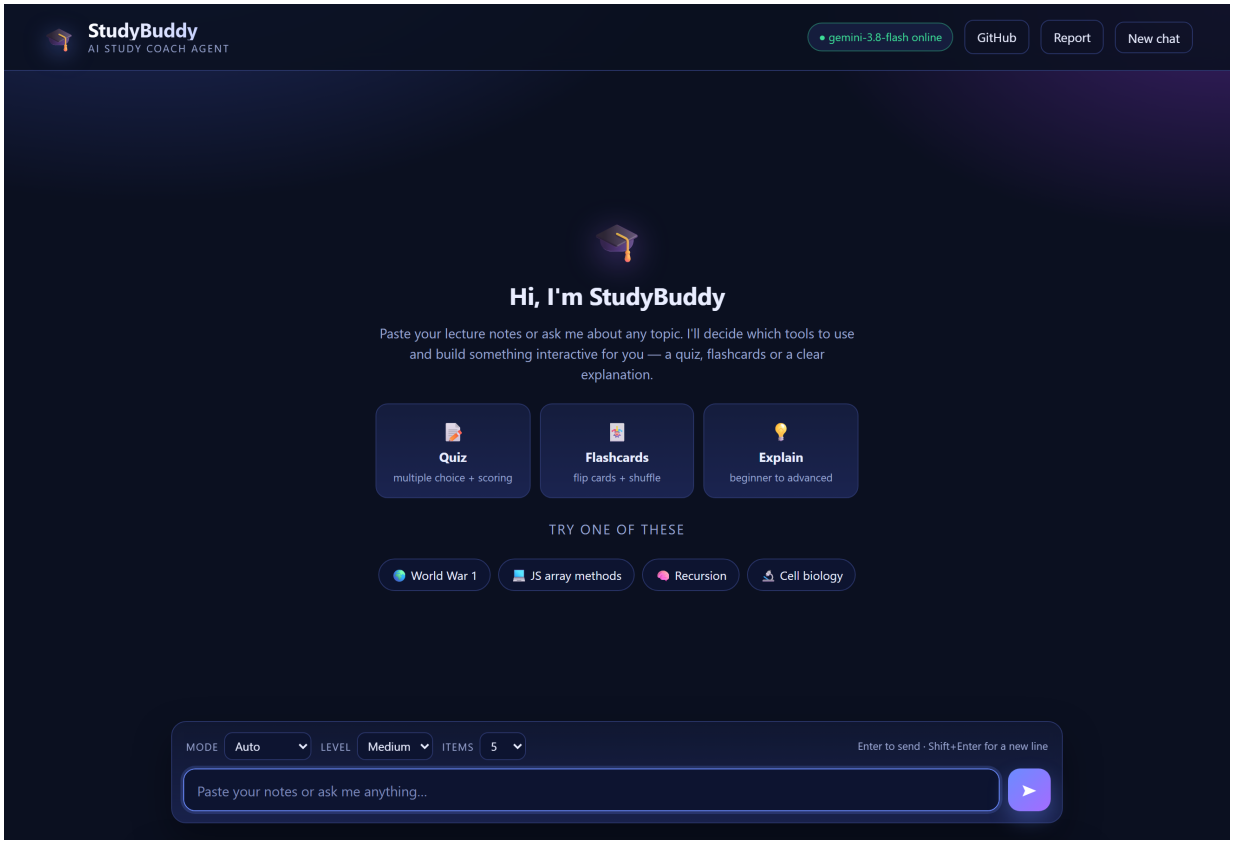



Figure 2: Landing screen: capability overview, example prompts and the composer.

## 7 Deployment

The application is deployed on **Vercel**. Heroku, also listed in the brief, removed its free tier; Vercel's free plan serves the static front end from `public/` and executes the API as Node.js *serverless functions*. The handlers in `api/` are *the same modules* the local Express server mounts, so local development and production run one identical code path – only the runtime around them differs.

1. Push the repository to GitHub: https://github.com/manganjokean/studybuddy.
2. In Vercel: *Add New* → *Project* and import the repository. The framework preset is *Other*; `vercel.json` declares `public/` as the static output directory and caps each function at 30 s.
3. Under *Settings* → *Environment Variables*, add `GEMINI_API_KEY` with the secret value. This is the *only* place the key is stored outside the developer's machine.
4. Deploy. Every push to `main` triggers a new deployment, and the site is reachable at https://studybuddy-XXXX.vercel.app.
5. Locally, `npm start` runs the identical handlers through Express for development.

**Free-tier caveat.** Nothing sleeps between requests: static assets are served from a CDN and only the two functions cold-start, which adds well under a second to the first request. The configured 30-second function limit is comfortably above the agent's typical end-to-end latency (three to four model calls).

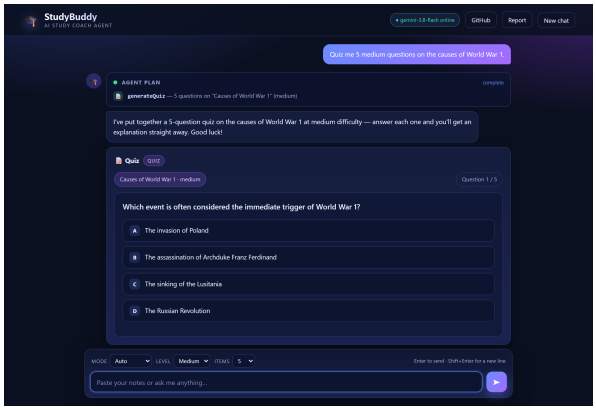



Figure 3: Chat turn: user request, agent plan and reply.

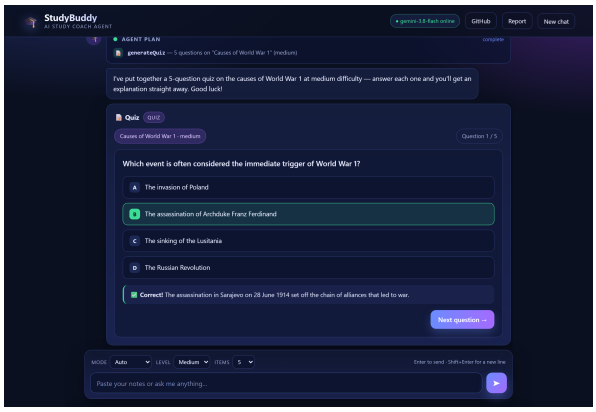



Figure 4: Inline quiz with per-question feedback.

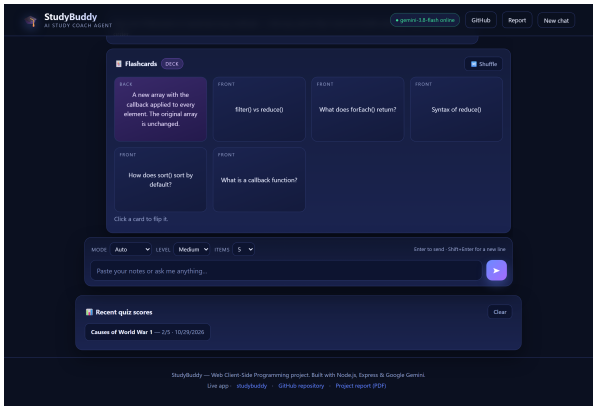



Figure 5: Flashcard deck with flip interaction.

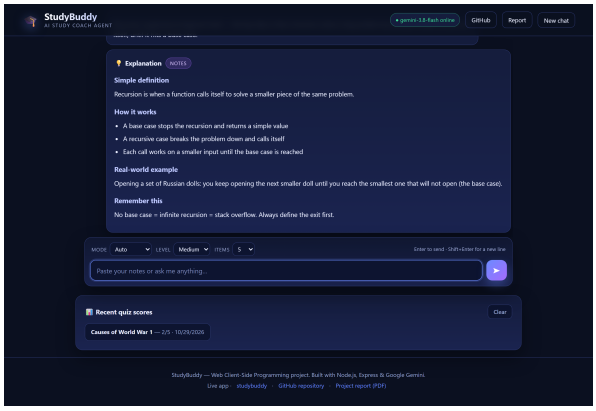



Figure 6: Structured concept explanation.

## 8 Security: API Key Handling

The brief explicitly forbids pushing API keys to GitHub. This was treated as a design constraint rather than an afterthought, and enforced at four levels:

Listing 2: Verifying the repository before every push.

```
$ npm run keycheck
> node scripts/keycheck.js
  No API keys or secrets found in the project. Safe to push.
```

Additionally, `.env.example` is committed with placeholder values only, which documents the required configuration without disclosing anything. The server also detects when the placeholder was never replaced and reports *"API key missing"* in the health endpoint instead of failing obscurely at request time.

## 9 Testing

## 10 Challenges and Lessons Learned

- **Express 5 breaking change.** The legacy wildcard route `app.get('*')` throws `Missing parameter name` under Express 5's `path-to-regexp` parser. The SPA fallback was rewritten

Table 4: Defence-in-depth for the secret.

| Layer | Mechanism |
|---|---|
| Repository | `.gitignore` written *before* the first commit; it lists `.env`, `.env.local`, `.env.*.local` and `node_modules/` |
| Code | The key is read only via `process.env.GEMINI_API_KEY`; there is no fallback literal anywhere in the source |
| Transport | The key is sent as the `x-goog-api-key` header directly to Google's API and is never returned in any HTTP response |
| Tooling | `npm run keycheck` scans every text file for credential patterns (Google `AIza`, OpenAI/Anthropic `sk-`, bearer tokens, assigned key variables) and exits non-zero if anything matches |

as a terminal `app.use()` handler.

- **Structured output is not optional.** Asking the model for free-form JSON failed intermittently; requesting `responseMimeType` plus an explicit schema, and then validating every field, removed those failures almost entirely.
- **Counting API calls.** The naive loop cost three model round-trips per request, which is slow on a free tier. Structured results are therefore returned to the client directly from the tool, and only the short conversational summary goes back through the model.
- **Secrets discipline.** Writing `.gitignore` first and adding an automated scanner turned "don't leak the key" from a reminder into an enforced rule.
- **Free-tier realities.** Cold starts and rate limits must be planned for when the application is demonstrated live.

## 11 Conclusion

StudyBuddy meets every requirement of the brief: it is a working web application built with a JavaScript framework-free client and an Express back end, it uses an AI agent with a genuine tool-calling loop, it is deployed publicly, it keeps its API key out of GitHub by construction, and this report links to both the live application and the repository.

The most valuable outcome was understanding the difference between a chatbot and an agent: giving the model *actions* to perform, and validating what comes back, is what allowed a single conversation to produce interactive artefacts the browser could render. Future work would add spaced-repetition scheduling for flashcards, server-side persistence of study history, and streaming responses so the plan trace appears in real time.

## Links

- Live application: https://studybuddy-XXXX.vercel.app
- Source repository: https://github.com/manganjokean/studybuddy

Table 5: Test cases executed before submission.

| Area | Case | Result |
|---|---|---|
| Health | `GET /api/health` reports model and key status | Pass |
| Validation | Empty message returns `400` with a clear error | Pass |
| Validation | Oversized message rejected (`413`) | Pass |
| Errors | Malformed JSON body returns `400`, not `500` | Pass |
| Errors | Missing key returns an actionable message | Pass |
| Errors | Rate limit (HTTP 429) mapped to friendly text | Pass |
| Routing | Unknown `/api` route returns JSON `404` | Pass |
| Routing | Deep links fall back to the single-page app | Pass |
| Agent | Quiz generated, validated and rendered interactively | Pass |
| Agent | Flashcards flip, shuffle and re-render | Pass |
| Agent | Score written to and read from `localStorage` | Pass |
| Security | `git ls-files` contains no `.env` | Pass |

- This report (PDF): `report/main.pdf`

## References


[1] Google AI for Developers, *Generating content — Gemini API*, https://ai.google.dev/api/generate-content, accessed 2026.

[2] ExpressJS, *Express 5.x Guide*, https://expressjs.com/, accessed 2026.

[3] Vercel, *Documentation — Node.js Functions*, https://vercel.com/docs/functions/runtimes/node-js, accessed 2026.

[4] S. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023.

[5] N. C. Dingemanse, "Retrieval practice is more effective than repeated study," *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 2020.